

AI WEATHERWISE

AI-Based Weather Information and Recommendation System using API

NAAN MUDHALVAN PROJECT

PROJECT TEAM DETAILS

TEAM LEADER	Vikashini.D
EMAIL ID	vikashini220307@gmail.com

TEAM MEMBERS

No.	Name	Email ID
1	Deepika	deepikadeepikadeepika0822@gmail.com
2	Manimegalai	mmani469769@gmail.com
3	Yuvamala	yuvamala025@gmail.com
4	Srimathi	srimathisri901@gmail.com
5		

Project Type: Web-based AI Weather Application

Technologies: Python • Flask • HTML • CSS • JavaScript • REST API • SQLite

Project Documentation

1. ABSTRACT

AI WeatherWise is a web-based weather application designed to provide useful weather information to users through a simple and user-friendly interface. The system uses a Weather API to obtain current weather information for a selected city. Python Flask is used as the backend framework, while HTML, CSS and JavaScript are used to create the frontend.

The application displays temperature, humidity, wind speed, pressure, weather condition and feels-like temperature. A lightweight AI-based rule analysis module examines the weather values and provides practical recommendations. For example, if the temperature is high, the system can suggest staying hydrated; if rain is detected, it can suggest carrying an umbrella.

2. OBJECTIVES

- To obtain current weather information using an external API.
- To develop a simple weather application using Python Flask.
- To connect frontend and backend using HTTP requests.
- To apply basic AI/rule-based analysis to weather data.
- To store recent searches using SQLite database.
- To present weather information in an easy-to-understand format.

3. KEY FEATURES

- City-based weather search.
- Current temperature, humidity and wind information.
- Weather condition and pressure display.
- AI-based weather recommendations.
- Search history storage.
- Responsive web interface.

4. SYSTEM REQUIREMENTS

Software Requirements

- Python 3.x
- Flask framework
- Requests library
- python-dotenv
- SQLite
- HTML5, CSS3 and JavaScript
- VS Code or PyCharm
- Modern web browser

Hardware Requirements

- Computer or laptop with at least 4 GB RAM.
- Internet connection for Weather API requests.
- Basic storage space for Python packages and project files.

5. TECHNOLOGY DESCRIPTION

Python: Used for backend programming, API processing and weather analysis.

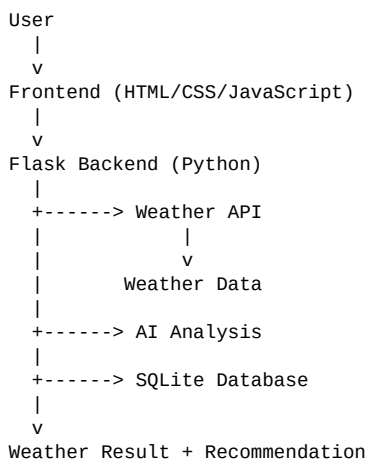
Flask: A lightweight Python web framework used to create backend routes and connect the frontend with the weather service.

Weather API: An external REST service that supplies current weather data for a requested city.

SQLite: A lightweight database used to store weather search history.

HTML/CSS/JavaScript: Used to build the user interface and dynamically display API results.

6. SYSTEM ARCHITECTURE



7. PROJECT MODULES

Module 1 – User Interface

The user enters a city name in the search box. JavaScript sends the request to the Flask backend.

Module 2 – Weather API Integration

The Flask backend sends the city name and API key to the weather service. The returned JSON data is converted into a simple Python dictionary.

Module 3 – Weather Processing

The application extracts temperature, humidity, pressure, wind speed, weather condition and other required values.

Module 4 – AI Weather Analysis

The project applies simple rule-based intelligence to interpret weather conditions and generate recommendations. This makes the raw API information more useful to the user.

Module 5 – Database

SQLite stores recent searches, including city, temperature, humidity, condition and search time.

Module 6 – Result Display

The final weather information and recommendations are displayed on the web page using JavaScript.

8. DATA FLOW

1. User enters city.
2. Frontend sends request to /api/weather.
3. Flask receives city name.
4. Flask calls Weather API.
5. API returns JSON weather data.
6. Python processes the data.
7. AI analysis generates recommendations.
8. Data is saved in SQLite.
9. Backend returns JSON result.
10. Frontend displays the result.

9. IMPORTANT BACKEND CODE

Flask route used to obtain weather information:

```
@app.route("/api/weather")
def weather():
    city = request.args.get("city")

    if not city:
        return jsonify({"error": "Please enter a city name"}), 400

    weather_data = get_weather(city)

    if "error" in weather_data:
        return jsonify(weather_data), 400

    analysis = analyze_weather(weather_data)
    save_weather(weather_data)

    return jsonify({
        "weather": weather_data,
        "analysis": analysis
    })
```

AI Analysis Logic

```
if temperature >= 35:
    message = "It is very hot today."
    recommendation = "Drink plenty of water."

elif temperature >= 30:
    message = "The temperature is warm."
    recommendation = "Stay hydrated."

elif temperature >= 20:
    message = "The temperature is comfortable."
    recommendation = "Normal outdoor activities are suitable."

else:
    message = "The temperature is cool."
    recommendation = "Carry a light jacket."
```

10. DATABASE STRUCTURE

Table: weather_history

Field	Type	Purpose
id	INTEGER	Unique record ID
city	TEXT	Searched city
temperature	REAL	Temperature in Celsius
humidity	INTEGER	Humidity percentage
condition	TEXT	Weather condition
searched_at	TIMESTAMP	Search date and time

11. API WORKING PROCESS

The Weather API works through HTTP requests. When the user enters a city, the backend creates an API request containing the city name, API key and metric unit. The API responds with JSON data. The application reads the required fields from that response.

Example API response fields used by the application:

```
{
  "main": {
    "temp": 30.5,
    "feels_like": 32.1,
    "humidity": 68,
    "pressure": 1008
  },
  "wind": {
    "speed": 3.5
  },
  "weather": [
    {
      "main": "Clouds",
      "description": "scattered clouds"
    }
  ]
}
```

12. FRONTEND WORKING

The frontend contains a city search box, search button, weather cards, AI analysis section and search history section. JavaScript uses the fetch() function to communicate with the Flask API without reloading the entire page.

13. EXPECTED OUTPUT

- City name and country are displayed.
- Temperature is shown in Celsius.
- Humidity is shown as a percentage.
- Wind speed is displayed in metres per second.
- Weather condition is shown.
- AI analysis provides understandable recommendations.
- Recent searches can be viewed from the SQLite database.

14. HOW TO RUN THE PROJECT

1. Open the project folder in VS Code/PyCharm.
2. Create and activate a Python virtual environment.
3. Install packages:
pip install -r requirements.txt
4. Add the Weather API key in .env.
5. Run:
python app.py
6. Open in browser:
http://127.0.0.1:5000

15. ADVANTAGES

- Easy to use for beginners.
- Provides current weather information through an API.
- AI-based recommendations make the data easier to understand.
- Uses a lightweight SQLite database.
- Can be expanded with more weather features.
- Demonstrates real-world API integration.

16. LIMITATIONS

- The application depends on internet connectivity.
- Weather accuracy depends on the external API and its data sources.
- An API key may have usage limits depending on the service plan.
- The AI module uses basic rules rather than a trained predictive model.
- Forecast features require additional API data and implementation.

17. FUTURE ENHANCEMENTS

- Add 5-day or 7-day weather forecasts.
- Add GPS/location-based weather search.
- Add weather maps and charts.
- Use a trained machine-learning model for prediction.
- Add severe-weather alerts.
- Add user accounts and personalized preferences.
- Deploy the application online using a cloud platform.

18. CONCLUSION

AI WeatherWise demonstrates how a real-world web application can combine Python, Flask, REST API integration, database storage and basic AI analysis. The system converts weather data into simple information and recommendations that are easy for users to understand. The project also provides practical experience in frontend development, backend development, API handling and database management.

19. VIVA QUESTIONS – SAMPLE

What is an API? API is an interface that allows one software application to communicate with another.

Why is Flask used? Flask is used to create the Python backend and API routes.

Why is SQLite used? SQLite provides simple local database storage.

What is the role of AI? AI-based rules interpret weather values and generate recommendations.

What happens when a city is searched? The frontend sends the city to Flask, Flask calls the Weather API, processes the response and returns the result.

End of Project Documentation